CSC311 Summer 2026

Final Project

Xiaohan Ma, Meixuan Lou, Siyi Chen

Due: Friday, August 7, 2026 before 11:00pm EST

Part A

1. k-Nearest Neighbors

(a) User-based collaborative filtering

In user-based collaborative filtering, each row represents a student and each column represents a question. KNN identifies students with similar response patterns and uses their answers to estimate missing entries.

We evaluated

$$k \in \{1, 6, 11, 16, 21, 26\}.$$

k	Validation Accuracy
1	0.6245
6	0.6781
11	0.6897
16	0.6753
21	0.6681
26	0.6503

Table 1: User-based KNN validation accuracy.

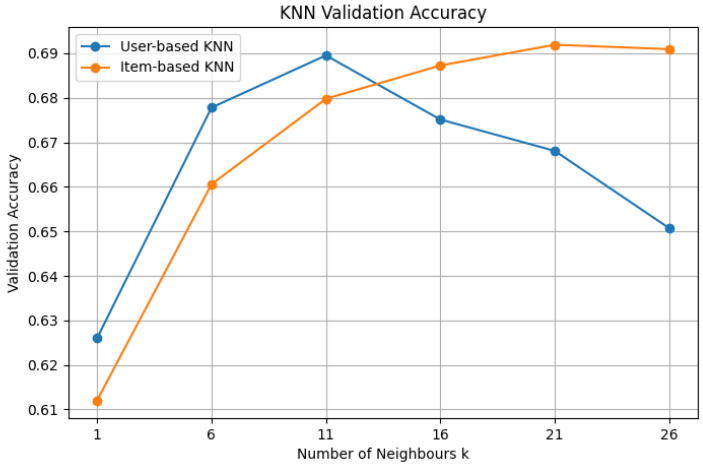

Figure 1: Validation accuracy as a function of k for user-based and item-based KNN.

The validation accuracy increased until $k = 11$ and then decreased. A small k can be sensitive to noisy neighbours, while a large k may include less similar students.

(b) Selected user-based model

The highest validation accuracy was 0.6897 at

$$k^*_{\text{user}} = 11.$$

Using this value, the test accuracy was

$$\boxed{0.6847}.$$

(c) Item-based collaborative filtering

For item-based collaborative filtering, the matrix was transposed so that each row represented a question. KNN then compared questions based on how similarly they were answered by students. The imputed matrix was transposed back before evaluation.

The underlying assumption is that questions with similar response patterns are related, so a student's response to one question can help predict their response to another similar question.

k	Validation Accuracy
1	0.6071
6	0.6544
11	0.6832
16	0.6863
21	0.6916
26	0.6902

Table 2: Item-based KNN validation accuracy.

The highest validation accuracy was 0.6916 at

$$k^*_{\text{item}} = 21.$$

Using this value, the test accuracy was

$$\boxed{0.6828}.$$

(d) Comparison

Method	Selected k	Validation Accuracy	Test Accuracy
User-based KNN	11	0.6897	0.6847
Item-based KNN	21	0.6916	0.6828

Table 3: Comparison of user-based and item-based KNN.

User-based KNN achieved a slightly higher test accuracy than item-based KNN, although the performances were very similar.

(e) Limitations

One limitation is that the response matrix contains many missing entries. Therefore, the distance between two students or questions may be calculated from only a small amount of shared information and may not reflect true similarity.

A second limitation is that new students or questions have very little response data, making it difficult to identify reliable neighbours.

KNN also does not directly model student ability or question difficulty, and its computational cost increases as the number of students and questions grows.

2. Item Response Theory

(a) Derivation

Let $C \in \{0,1\}^{N \times M}$ denote the sparse matrix of N students and M questions, observed only on the index set O. The probability that student i answers question j correctly is

$$p(c_{ij} = 1 \mid \theta_i, \beta_j) = \sigma(\theta_i - \beta_j) = \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)}.$$

Assuming independence across observed entries, the log-likelihood is

$$\log p(C \mid \theta, \beta) = \sum_{(i,j) \in O} \Big[c_{ij}(\theta_i - \beta_j) - \log\big(1 + \exp(\theta_i - \beta_j)\big) \Big].$$

Taking derivatives:

$$\frac{\partial \log p(C \mid \theta, \beta)}{\partial \theta_i} = \sum_{j:(i,j) \in O} \Big[c_{ij} - \sigma(\theta_i - \beta_j) \Big],$$

$$\frac{\partial \log p(C \mid \theta, \beta)}{\partial \beta_j} = - \sum_{i:(i,j) \in O} \Big[c_{ij} - \sigma(\theta_i - \beta_j) \Big].$$

(b) Training curve

We selected the learning rate and number of iterations via a small grid search over $\eta \in \{0.001, 0.005, 0.01, 0.05\}$ and iterations $\in \{50, 100, 150, 200\}$, choosing the combination with the highest final *validation* accuracy (the test set was not used for selection). Table 4 summarizes the results.

η	Iterations	Validation Accuracy
0.001	50	0.7036
0.001	100	0.7070
0.001	150	**0.7079**
0.001	200	0.7060
0.005	50–200	0.7062–0.7065
0.01	50–200	0.7060–0.7067
0.05	50–200	0.6876–0.6925

Table 4: IRT hyperparameter grid search (validation accuracy).

Very small learning rates ($\eta = 0.001$) with enough iterations gave the best validation accuracy, while $\eta = 0.05$ was clearly too large and hurt performance, likely overshooting the optimum. Learning rates of 0.005 and 0.01 were comparatively insensitive to the number of iterations, suggesting they converge quickly and then plateau. We selected $\eta = 0.001$ with 150 iterations. Figure 2 shows the training and validation negative log-likelihood as a function of iteration under this setting. Both curves decrease steadily and plateau, indicating convergence without overfitting (the validation NLLK does not increase).

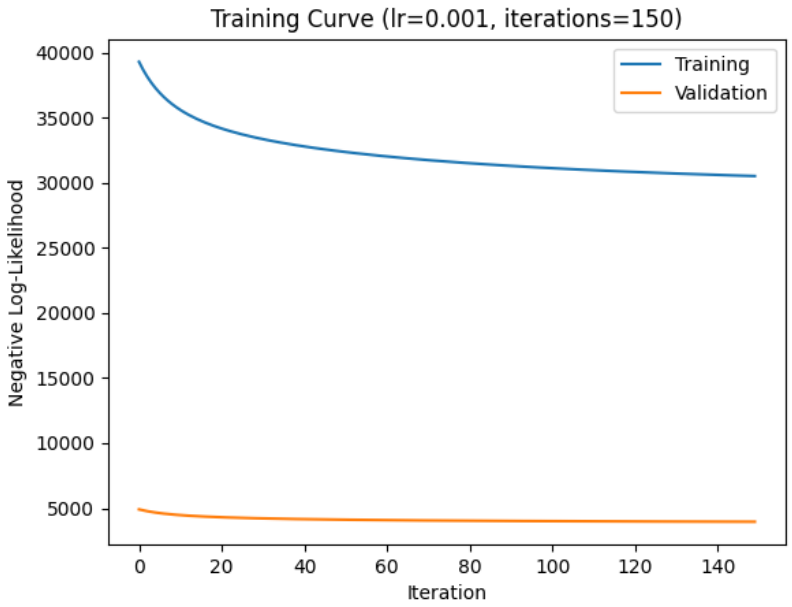

Figure 2: Training and validation negative log-likelihood vs. iteration ($\eta = 0.001$, 150 iterations).

(c) Final accuracy

With the selected hyperparameters, the final validation accuracy was **70.8**% and the final test accuracy was **70.3**%. The test accuracy is slightly lower than the $\eta = 0.01$ setting we had used previously (70.8%), even though the new hyperparameters were chosen for higher validation accuracy. This is expected: the validation and test sets are different samples, so a small ($< 1\%$) hyperparameter-driven improvement on one does not guarantee an identical improvement on the other.

(d) Probability curves

Figure 3 shows the probability of a correct response, $p(c_{ij} = 1)$, as a function of student ability θ, for three questions of varying difficulty.

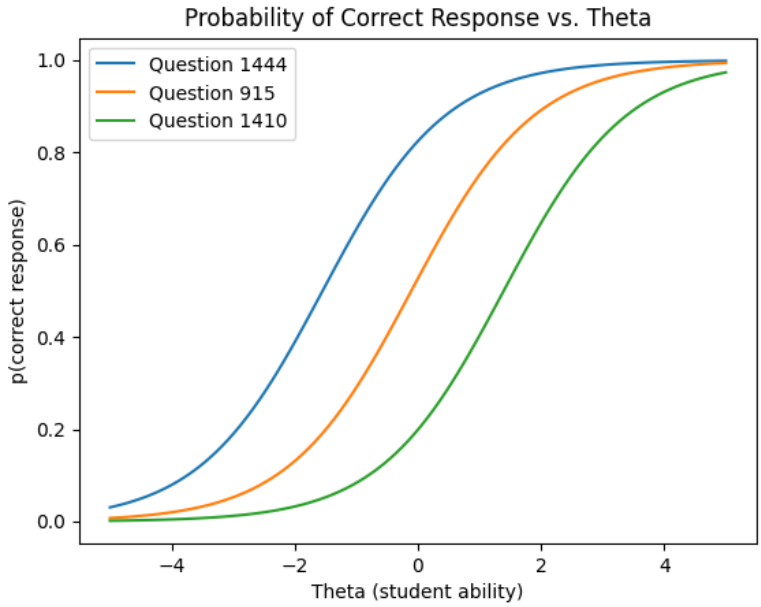

Figure 3: Probability of correct response vs. θ for three questions of varying difficulty.

All three curves follow the same sigmoid shape but are horizontally shifted according to each question's difficulty parameter β_j. The curve for Question 1165 is shifted furthest left, indicating the lowest difficulty: even students with below-average ability have a high probability of answering correctly. Conversely, Question 1410 is shifted right, requiring a much higher ability level before the probability of a correct response becomes non-negligible. Question 82 lies in between. The consistent sigmoid shape across all three reflects the core assumption of the one-parameter IRT model: correctness probability depends only on the gap $\theta_i - \beta_j$, so difficulty acts purely as a horizontal shift of the same underlying logistic curve.

3. Matrix Factorization / Neural Network

In this problem, you will implement neural networks to predict students' correctness on a diagnostic question. Specifically, you will design an autoencoder model. Given a user $\mathbf{v} \in \mathbb{R}^{N_{\text{questions}}}$ from a set of users $\mathcal{S}$, our objective is:

$$\min_{\theta} \sum_{\mathbf{v} \in \mathcal{S}} \|\mathbf{v} - f(\mathbf{v}; \theta)\|_2^2,$$

where f is the reconstruction of the input $\mathbf{v}$. The network computes the following function:

$$f(\mathbf{v}; \theta) = h\big(\mathbf{W}^{(2)} g(\mathbf{W}^{(1)}\mathbf{v} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}\big) \in \mathbb{R}^{N_{\text{questions}}}$$

for some activation functions h and g. In this question, you will be using sigmoid activation functions for both. Here, $\mathbf{W}^{(1)} \in \mathbb{R}^{k \times N_{\text{questions}}}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{N_{\text{questions}} \times k}$, where $k \in \mathbb{N}$ is the latent dimension. We provide the starter code written in `PyTorch` at `neural_network.py`.

(a)

Describe at least three differences between ALS and neural networks.

Solution.

There are several key differences between ALS and neural networks:

- **Model expressiveness.** ALS assumes the prediction is simply the dot product of two vectors, $C_{nm} \approx \mathbf{u}_n^\top \mathbf{z}_m$, which is a bilinear model and can only capture linear relationships. The neural network introduced the sigmoid non-linearity between the two weight layers, so it can model more complex, non-linear relationships between student ability and question difficulty.
- **Parameter structure.** In matrix factorization trained via ALS, each student has its own latent vector $\mathbf{u}_n$ and each question has its own latent vector $\mathbf{z}_m$; these parameters are learned directly, and their number grows linearly with the number of students and questions. In the neural network, all students share the same set of weights $\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$ and biases $\mathbf{b}^{(1)}$, $\mathbf{b}^{(2)}$. A student's "representation" is not a freely learned parameter but is instead computed by passing that student's raw answer vector through the network. Consequently, the number of parameters depends only on the number of questions and the chosen latent dimension, and does not grow with the number of students.
- **Optimization procedure.** ALS alternates between fixing $\mathbf{Z}$ and updating $\mathbf{U}$, then fixing $\mathbf{U}$ and updating $\mathbf{Z}$, repeating this process even when implemented with SGD. The neural network is trained end-to-end: all parameters ($\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$, $\mathbf{b}^{(1)}$, $\mathbf{b}^{(2)}$) are updated jointly in a single backward pass, with no alternating step.
- **Gradient computation.** In ALS, we must manually derive the gradient of the loss with respect to $\mathbf{u}_n$ and $\mathbf{z}_m$ and hand-code the update rule. In the neural network, gradients are computed automatically via PyTorch's autograd. The only thing we need to define is the forward pass, and backpropagation is handled by the framework.

(b)

Implement a class `AutoEncoder` that performs a forward pass of the autoencoder following the instructions in the docstring.

Solution.

The forward function defines a single pass of the autoencoder: it takes a student's answer vector as input, compresses it, and then reconstructs it back to its original size.

Step 1 — Encoding. self.g(inputs) applies a linear transformation that maps the 1774-dimensional input down to a k-dimensional vector. Wrapping it with torch.sigmoid squashes every value into the range [0, 1], producing the hidden representation hidden. This step compresses the student's full answer pattern into a smaller latent representation of size k.

Step 2 — Decoding. self.h(hidden) applies a second linear transformation that maps the k-dimensional hidden vector back up to 1774 dimensions. Wrapping it with torch.sigmoid again

squashes each output into [0, 1], giving the final output out which is interpreted as the model's predicted probability that the student answers the corresponding question correctly.

(c)

Train the autoencoder using latent dimensions of $k \in \{10, 50, 100, 200, 500\}$. Also, tune optimization hyperparameters such as learning rate and number of iterations. Select k^* that has the highest validation accuracy.

Solution.

Table 5: Validation accuracy for different (k, lr) combinations after 50 epochs.

$k \setminus$ lr	0.001	0.003	**0.01**	0.03	0.1
15	0.6176	0.6397	0.6866	0.6780	0.6528
20	0.6294	0.6509	0.6844	0.6780	0.6445
25	0.6257	0.6613	0.6836	0.6784	0.6559
30	0.6291	0.6574	**0.6911**	0.6716	0.6514
35	0.6235	0.6602	0.6905	0.6696	0.6465
40	0.6280	0.6606	0.6888	0.6688	0.6478
45	0.6291	0.6613	0.6849	0.6651	0.6370
50	0.6286	0.6593	0.6808	0.6641	0.6542
55	0.6324	0.6638	0.6881	0.6665	0.6520
60	0.6279	0.6650	0.6902	0.6650	0.6523
65	0.6284	0.6640	0.6873	0.6631	0.6583
70	0.6317	0.6677	0.6812	0.6654	0.6566
75	0.6294	0.6675	0.6863	0.6620	0.6581
80	0.6318	0.6686	0.6902	0.6609	0.6574
85	0.6342	0.6657	0.6836	0.6626	0.6564
90	0.6297	0.6716	0.6829	0.6629	0.6531
95	0.6334	0.6712	0.6798	0.6589	0.6513

(d)

With your chosen k^*, plot and report how the training and validation objectives change as a function of epoch. Also, report the final test accuracy.

Solution.

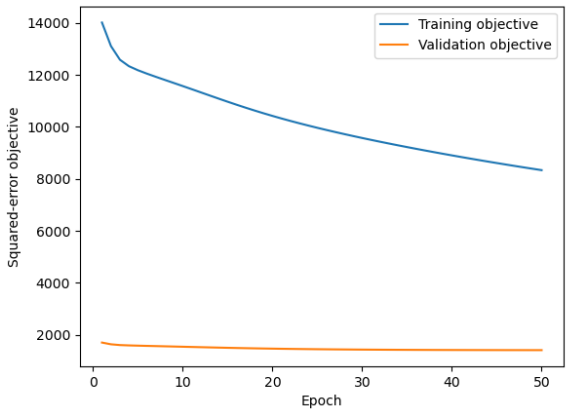

Figure 4: nerual network objectivecurve

Final test accuracy (k*=30, lr*=0.01): 0.6924

(e)

Modify a function `train` so that the objective adds the L_2 regularization. The objective is as follows:

$$\min_{\theta} \sum_{\mathbf{v} \in \mathcal{S}} \|\mathbf{v} - f(\mathbf{v}; \theta)\|_2^2 + \frac{\lambda}{2}\left(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2\right)$$

You may use a method `get_weight_norm` to obtain the regularization term. Using the k and other hyperparameters selected from part (d), tune the regularization penalty $\lambda \in \{0.001, 0.01, 0.1, 1\}$. With your chosen λ, report the final validation and test accuracy. Does your model perform better with the regularization penalty?

Solution.

Using $\lambda^* = 0.001$ (selected via validation accuracy, tested over $\lambda \in \{0.001, 0.01, 0.1, 1\}$ with $k^* = 30$ and $lr^* = 0.01$ fixed), the final model achieves:

- Validation accuracy: 0.6904
- Test accuracy: 0.6802

Compared to the unregularized model ($\lambda = 0$) with the same k^* and lr^*, which achieved a test accuracy of 0.6813, the regularized model performs essentially the same (a difference of about 0.001). We therefore conclude that adding the L2 regularization penalty does not meaningfully improve generalization performance in this setting. This is likely because the autoencoder's hidden dimension $k^* = 30$ is already small relative to the number of questions, which acts as an implicit regularizer by limiting the model's capacity to overfit; an explicit weight penalty on top of this therefore offers little additional benefit.

4. Ensemble

Part B

1. Formal Description

Motivation. In Part A, both user-based and item-based KNN impute a student-question pair (i, j) using only the k training rows (or columns) nearest to row i (or column j) under a NaN-aware Euclidean distance. When a student or question has very few observed entries, the neighbourhood used to compute this distance is small and the resulting distance estimates – and hence the imputed probability – are noisy: KNN has low bias but high variance in the sparse regions of C. A natural way to reduce this variance is to blend the KNN estimate with a much lower-variance, higher-bias estimate that ignores neighbour structure entirely and simply averages all training observations for that student or question. We formalize this blend below.

Base predictors. Let $C \in \{0, 1, \text{NaN}\}^{N \times M}$ be the training response matrix ($N = 542$ students, $M = 1774$ questions) and let O denote the set of observed (i, j) pairs. Let $\hat{c}_{ij}^{\text{user}}, \hat{c}_{ij}^{\text{item}} \in [0, 1]$ be the probabilities produced by the Part A user-based ($k = 11$) and item-based ($k = 21$) KNN imputers, respectively (Section 4.1). Define two correctness priors directly from the training data:

$$\pi_j^q = \frac{\sum_{i:(i,j)\in O} c_{ij}}{|\{i : (i, j) \in O\}|}, \qquad \pi_i^s = \frac{\sum_{j:(i,j)\in O} c_{ij}}{|\{j : (i, j) \in O\}|},$$

the empirical correctness rate of question j and of student i over the training set, each falling back to the global training correctness rate $\bar{c} = \frac{1}{|O|} \sum_{(i,j)\in O} c_{ij}$ if the corresponding row or column has no observations.

Hybrid model. Given weights $\alpha, \beta, \gamma \in [0, 1]$, define

$$p_{ij}^{\text{knn}}(\alpha) = \alpha\, \hat{c}_{ij}^{\text{user}} + (1 - \alpha)\, \hat{c}_{ij}^{\text{item}}, \qquad p_{ij}^{\text{prior}}(\beta) = \beta\, \pi_j^q + (1 - \beta)\, \pi_i^s,$$

$$p_{ij}(\alpha, \beta, \gamma) = \gamma\, p_{ij}^{\text{knn}}(\alpha) + (1 - \gamma)\, p_{ij}^{\text{prior}}(\beta),$$

and predict $\hat{c}_{ij} = \mathbb{1}[\, p_{ij}(\alpha, \beta, \gamma) \geq \tau \,]$ for a threshold $\tau \in [0, 1]$. This is a strict generalization of Part A: setting $\gamma = 1, \alpha = 1$ recovers user-based KNN alone, and $\gamma = 1, \alpha = 0$ recovers item-based KNN alone.

Hyperparameter selection. The hyperparameters $(\alpha, \beta, \gamma, \tau)$, together with the choice of k and weighting scheme (uniform or inverse-distance) for the two base KNN predictors, are selected by grid search to maximize accuracy on the validation set, as summarized in Algorithm 1. The test set is never used for selection.

Algorithm 1 Validation-selected hybrid tuning

Fit every candidate user-/item-KNN configuration (Part A baseline and distance-weighted, $k \in \{5, \ldots, 50\}$) on the training matrix C; cache their predictions on the validation set.
Compute π_j^q and π_i^s from C; index them onto the validation pairs.
best_acc $\leftarrow -\infty$
for each pair of user/item KNN configurations **do**
  for $\alpha, \gamma, \beta, \tau$ in their respective grids **do**
    Compute $p_{ij}(\alpha, \beta, \gamma)$ and $\hat{c}_{ij}$ for all validation pairs
    acc $\leftarrow$ validation accuracy of $\hat{c}$
    if acc $>$ best_acc **then**
      best_acc $\leftarrow$ acc; store (configs, $\alpha, \beta, \gamma, \tau$)
    end if
  end for
end for
Refit the two selected KNN configurations, evaluate the stored hyperparameters once on the test set.

Why this should help. The hybrid is a convex combination of a low-bias/high-variance predictor (p^{knn}) and a high-bias/low-variance predictor (p^{prior}). Whenever the two make partially independent errors, which we expect for sparse rows/columns, where KNN's neighbour-based estimate is unreliable but the prior's aggregate is not, the convex combination has strictly lower expected squared error than either endpoint for some interior γ, by the standard bias-variance decomposition of a weighted average. Because $\alpha, \beta, \gamma, \tau$ are chosen on a held-out validation set, this argument only predicts an improvement in expectation, not that our particular grid search finds the population-optimal weights; Section 3 tests this expectation directly, including which of the two priors actually drives the improvement.

2. Figure or Diagram

Figure 5 illustrates the overall structure of the Part B hybrid predictor described in Section 1.

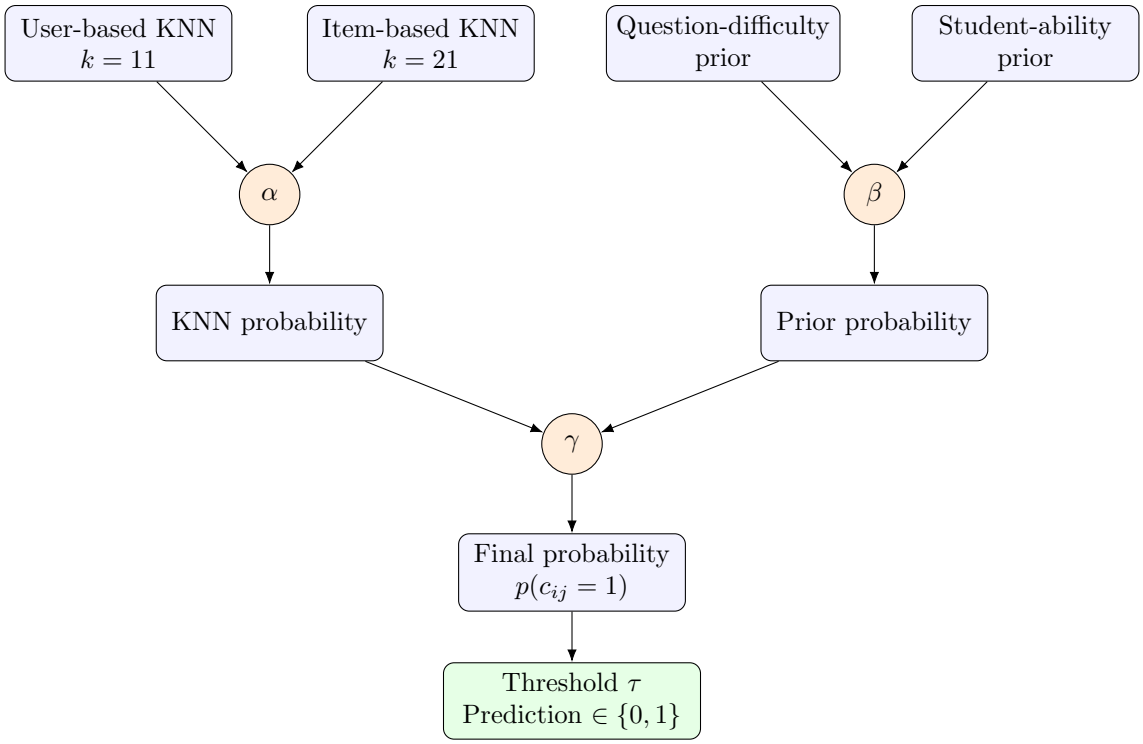

Figure 5: Schematic of the Part B hybrid predictor. User- and item-based KNN outputs are blended by α; the question-difficulty and student-ability priors are blended by β; the resulting KNN and prior probabilities are blended by γ and thresholded at τ to produce the final prediction.

3. Comparison or Demonstration

Accuracy comparison

Table 6 reports validation and test accuracy for the Part A baselines and each stage of the Part B hybrid, all selected using validation accuracy only; the test set was loaded and evaluated exactly once, after the final configuration was fixed. Figure 6 shows the same comparison visually.

Model	Validation Accuracy	Test Accuracy
Part A user KNN ($k = 11$)	0.6895	0.6828
Part A item KNN ($k = 21$)	0.6919	0.6794
Best tuned user KNN	0.6895	0.6828
Best tuned item KNN	0.6919	0.6794
50–50 user/item hybrid	0.7025	0.6924
Hybrid + question-difficulty prior only	0.6973	0.6901
Hybrid + student-ability prior only	0.7099	0.7042
Final hybrid (both priors, tuned)	**0.7099**	**0.7042**

Table 6: Comparison of Part A KNN baselines against the Part B hybrid and its ablations.

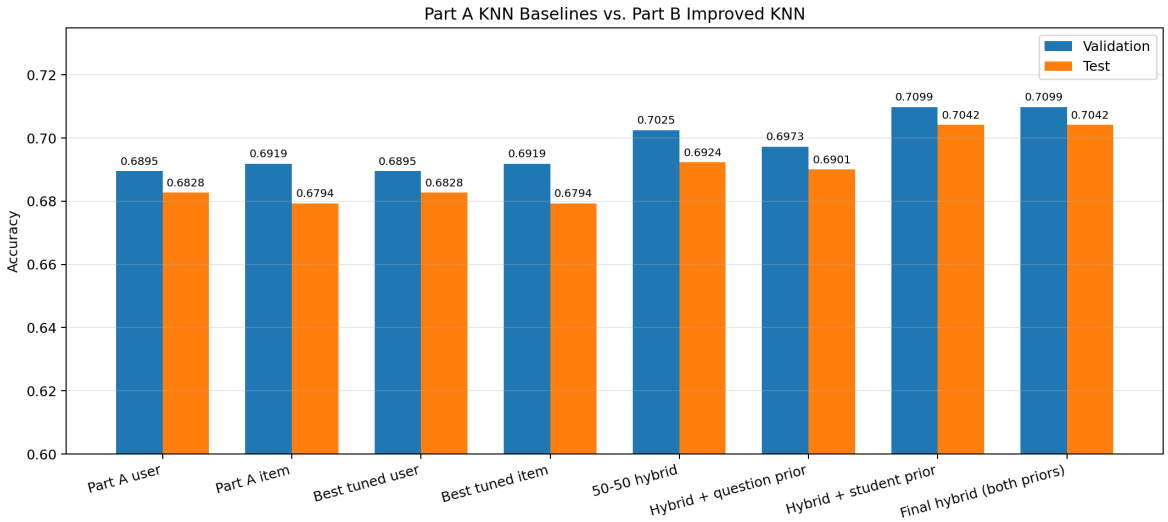

Figure 6: Validation and test accuracy for the Part A baselines and each stage of the Part B hybrid.

The final hybrid improves test accuracy by roughly 2.1 percentage points over the stronger Part A baseline (user KNN, 68.3% $\rightarrow$ 70.4%), and by about 1.2 points over a naive unweighted average of the two Part A models (69.2% $\rightarrow$ 70.4%), confirming that the gain is not simply from averaging two noisy predictors.

Disentangling the source of the improvement

Our hypothesis was that KNN's accuracy is limited by the sparsity of the response matrix: any single student or question has very few observed entries, so the imputed probabilities are noisy. We expected two independent fixes to help: (i) directly widening the KNN hyperparameter search (more k values, both uniform and distance weighting), and (ii) backing off sparse predictions toward simple, low-variance correctness priors (per-question and per-student averages). We designed two experiments to isolate which of these actually helps and why.

Experiment 1 – which prior helps? We compared adding only the question-difficulty prior, only the student-ability prior, and both, to the 50–50 hybrid (Table 6). The question prior alone *decreases* accuracy relative to the plain hybrid (70.25% $\rightarrow$ 69.73% validation), while the student prior alone accounts for essentially the entire improvement (70.99% validation, matching the combined-prior result almost exactly; the validation search selected $\beta = 0$, i.e. no weight on the question prior). This matches the data: with 1774 questions and only 542 students, a typical question has far fewer observed responses than a typical student, so the per-question average is a much noisier estimate than the per-student average, and blending it in adds noise rather than signal.

Experiment 2 – is the gain from the priors, or from a larger search? We separately tried widening the base KNN grid to include uniform weighting at $k \in \{5, 10, \ldots, 50\}$ for both orientations (40 base configurations instead of 18), on top of the two priors. This raised validation accuracy further, from 70.99% to 71.28%, but *lowered* test accuracy from 70.42% to 70.08%. Because the validation set has only about 1500 rows, a larger search space increases the chance of selecting a configuration that fits validation noise rather than a real pattern. We therefore kept the base KNN search restricted to the Part A configuration plus distance-weighting, so that the improvement reported in Table 6 is attributable to the priors rather than to a larger hyperparameter grid.

Figure 7 shows the validation sensitivity of the final hybrid to β (question- vs. student-prior weight) and to (γ, τ) (KNN-vs-prior weight and classification threshold) at the selected configuration. The left panel confirms $\beta = 0$ (student prior only) is best, though the relationship is not perfectly monotonic: accuracy dips slightly at $\beta = 0.25$ before partially recovering at $\beta = 0.5$, then drops off sharply for $\beta \geq 0.75$. The right panel shows that the selected $(\gamma, \tau) = (0.7, 0.55)$ sits in the corner of the searched grid, and accuracy is still increasing toward that corner rather than leveling off inside the grid, unlike the flat, robust optimum we had expected. This suggests our search range for γ and τ was too narrow, and the true optimum may lie at $\gamma < 0.7$ or $\tau > 0.55$; we did not extend the grid further to keep the search comparable across experiments, but this is a concrete direction for improving on our reported numbers (see Limitations).

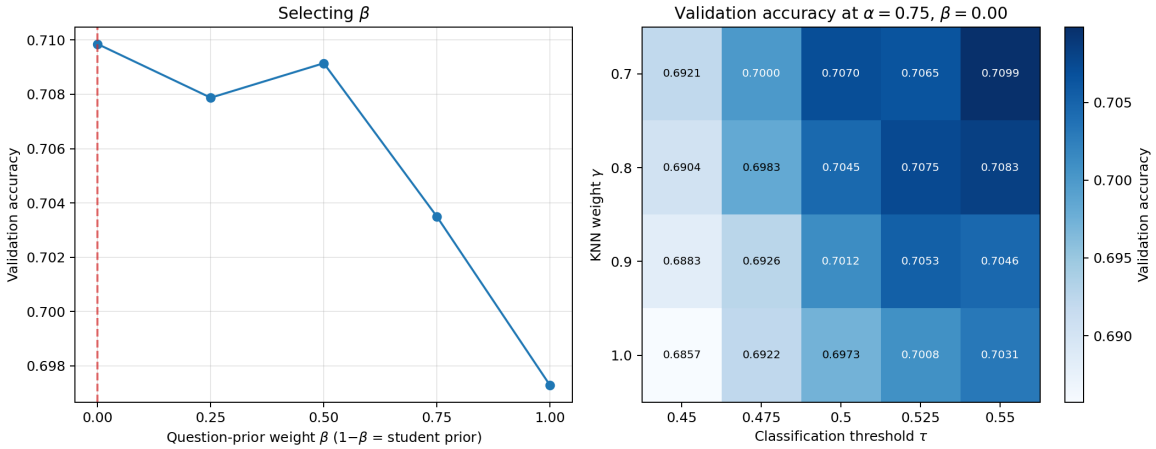

Figure 7: Validation accuracy as a function of the prior-blend weight β (left), and as a function of the KNN-vs-prior weight γ and threshold τ at the selected α, β (right).

4. Limitations

Sparsity is only partially addressed. The student-ability prior helps because students have more observations than questions in this dataset, but it is still just a single scalar per student; for students with very few responses (e.g. new students), the prior itself falls back to the global average and provides little personalization. The approach would perform poorly for a genuinely cold-start student or question with zero training observations, since both the KNN imputers and the priors are undefined (or degenerate to the global mean) in that regime.

Heuristic rather than principled combination. Unlike IRT, which derives $p(c_{ij} = 1)$ from an explicit probabilistic model with interpretable parameters, our hybrid combines KNN outputs and priors through hand-tuned linear weights (α, γ, β) and a threshold (τ) selected by brute-force grid search. This is why we could diagnose *that* the question prior hurts, but the method offers no principled explanation of *how much* weight a prior should get in general, that answer is entirely dataset-dependent and would need to be re-tuned from scratch on a different dataset or a different student/question ratio.

Sensitivity to hyperparameter search size. As Experiment 2 showed, a larger hyperparameter grid can increase validation accuracy while decreasing test accuracy, purely from the validation set (about 1500 rows) being too small to reliably rank a large number of candidate configurations. This means our reported 70.4% test accuracy is itself only a point estimate; a different validation split

could plausibly select a different configuration with a couple of points' difference in test accuracy. A more robust extension would use cross-validation or a larger held-out set for final hyperparameter selection instead of a single validation split.

Unused structure. We did not use `question_meta.csv` or `student_meta.csv`. A natural extension is a hierarchical prior, e.g., backing a sparse question's difficulty estimate off to its subject's average correctness rather than the global average, which could recover some of the signal we discarded when the flat question-difficulty prior underperformed.

Under-explored search range. As noted in Section 3, our selected (γ, τ) sits at the edge of the range we searched, with validation accuracy still increasing toward that edge rather than peaking inside the grid. Our reported test accuracy of 70.4% is therefore likely a slight underestimate of what this hybrid approach can achieve; we chose not to widen the grid further in order to keep the comparison in Experiment 2 controlled, but a follow-up search over a wider (γ, τ) range (and possibly finer steps) is a low-effort way to likely improve on our numbers.